Implementation of 1D and 2D Beamforming using Matlab and Xilinix Vivado

Project Report

Submitted by

Kothuru Kavya

Intern SPG division, Homing Department

Naval Science & Technological Laboratory (NSTL), DRDO

Visakhapatnam

Under the guidance of

Mr. CH.Challam Naidu Sir (Sc – E)

SPG division, Homing Dept.

Naval Science & Technological Laboratory (NSTL), DRDO

Visakhapatnam (A.P)-532007

4th May 2026-30th October 2026

DECLARATION

I, Kothuru Kavya, declare that the work presented in this report titled **"Implementation of 1D and 2D Beamforming Using MATLAB and Xilinx Vivado "** has been carried out by me during my internship at the SPG Division, Homing, Naval Science & Technology Laboratory (NSTL), (DRDO), Visakhapatnam.

This work reflects my own effort and understanding, developed under the guidance and support of **Mr. Cheepurapalli Challam Naidu (Sc - E) , SPG division, HOMING Dept., NSTL, DRDO**. The study, analysis, results, and explanations included in this report are based on the tasks and research I performed as a part of my internship.

I also confirm that this report is original and has not been submitted, either in full or in part, to any university, institute, or organization for any degree, certification, or academic requirement. Any external sources of information used in the preparation of this report have been properly acknowledged.

Kothuru Kavya
Intern
NSTL, DRDO

Project Guide:
CH.Challam Naidu
(Scientist-E), Homing Dept

Date: _____

Place: Visakhapatnam

ACKNOWLEDGEMENTS

I would like to take this opportunity to express my sincere gratitude to all those who supported and guided me throughout the course of my internship project.

I am deeply grateful to **Mr. Ch.Challam Naidu Sir** Scientist ‘E’, SPG Division,Homing Dept, NSTL-DRDO, for his invaluable guidance, encouragement, and insightful suggestions from the very early stage of this work. His constant support and expertise have been instrumental in shaping the direction and completion of this work. I would like to thank him for his constant encouragement that enabled me to grow as a person.

I express my gratitude towards Mrs. M. Vijaya Mam, OS Scientist-H, Technical Director, SPG Division, DRDO NSTL, for providing me with all the help and permitting me to work in the laboratory with no time limits.

I shall ever remain indebted to Dr. Abraham Varughese, Director, DRDO NSTL, for providing me with institutional and administrative facilities during my project work at DRDO NSTL

Kothuru Kavya

ABSTRACT

Beamforming is a digital signal processing technique used to enhance signals arriving from a desired direction. This internship focuses on the implementation and simulation of one-dimensional (1D) and two-dimensional (2D) beamforming techniques using MATLAB 2021 and Xilinx Vivado 2019.1 at DRDO, Naval Science and Technological Laboratory (NSTL).

The work includes sine wave generation using the DDS Compiler IP, delayed sine wave generation, and Linear Frequency Modulated (LFM) signal analysis using MATLAB. A 1D linear time-delay beamformer was studied, while a 2D planar hydrophone array was analyzed using time-delay calculations, array factor computation, normalized beam patterns, and azimuth and elevation cuts. Additionally, a 10× polyphase interpolation system using a 60th-order FIR filter was designed and simulated. Verilog RTL, COE files, and MEM files were used in the Vivado simulation workflow where applicable.

This internship provided practical knowledge of digital signal processing, beamforming, MATLAB-based analysis, Verilog RTL design, and simulation-based verification. The work establishes a foundation for future hardware implementation of 1D and 2D beamforming systems.

TABLE OF CONTENTS

INTRODUCTION

OBJECTIVES OF THE INTERNSHIP

1. CHAPTER 1: SOFTWARE TOOLS AND TECHNOLOGIES

1.1 MATLAB 2021
1.2 Xilinx Vivado 2019.1
1.3 Verilog Hardware Description Language
1.4 DDS Compiler IP
1.5 Block Memory Generator IP
1.6 COE and MEM Files
1.7 RTL Design and Simulation
1.8 MATLAB-to-Vivado Integration

2. CHAPTER 2: SINE WAVE GENERATION USING DDS COMPILER

2.1 Introduction
2.2 Principle of Direct Digital Synthesis
2.3 DDS Compiler IP Configuration
2.4 Design Methodology
2.5 Simulation Setup
2.6 Simulation Results and Waveform Analysis

3. CHAPTER 3: DELAYED SINE WAVE GENERATION

3.1 Introduction
3.2 Concept of Clock-Cycle Delay
3.3 Design Methodology
3.4 Verilog RTL Implementation
3.5 Simulation Setup
3.6 Output Waveform Analysis

4. CHAPTER 4: LFM SIGNAL GENERATION USING MATLAB

4.1 Introduction
4.2 Principle of Linear Frequency Modulation
4.3 Mathematical Representation of LFM Signal
4.4 MATLAB Implementation
4.5 Time-Domain Analysis
4.6 Frequency Characteristics
4.7 Results and Discussion

5. CHAPTER 5: LINEAR TIME-DELAY BEAMFORMER

5.1 Introduction
5.2 Basic Principle of Beamforming
5.3 Linear Time-Delay Beamforming Concept
5.4 Grating lobes concept
5.5 MATLAB Design and Implementation
5.6 Data Export Using COE Files
5.7 Vivado Integration and Simulation
5.8 Results and Discussion

6. CHAPTER 6: 10× POLYPHASE INTERPOLATION USING A 60TH-ORDER FIR FILTER

6.1 Introduction
6.2 Fundamentals of Interpolation
6.3 Polyphase Interpolation Principle
6.4 FIR Filter Design Using MATLAB
6.5 10× Interpolation Architecture
6.6 Exporting Filter Coefficients Using COE Files
6.7 Input Sample Initialization Using MEM Files
6.8 Vivado Simulation and Waveform Analysis
6.9 Results and Discussion

7. CHAPTER 7: PLANAR ARRAY BEAMFORMER

7.1 Introduction

7.2 Planar Array Geometry

7.3 Direction of Arrival (Azimuth and Elevation Angles)

7.4 Time-Delay Calculation

7.5 Conventional Beamforming Principle

7.6 Array Factor Calculation

7.7 Normalized Beam Pattern and Azimuth and Elevation Cuts

7.8 Hydrophone Spacing and Grating Lobes

7.9 MATLAB Analysis and Implementation

7.10 Results and Discussion

8. CHAPTER 8: CONCLUSION REFERENCES

1. INTRODUCTION

Beamforming is a digital signal processing technique used to control the directional response of an array of sensors. It combines signals received by multiple sensors by applying suitable time delays or phase shifts and then summing them to enhance signals arriving from a desired direction. Beamforming is widely used in radar, sonar, underwater acoustic communication, wireless communication, and sensor array processing.

In underwater acoustic systems, hydrophones are used to receive acoustic signals propagating through water. When a sound wave arrives at a hydrophone array, it reaches different hydrophones at different times because of the differences in their positions. By compensating for these time differences and combining the received signals, the beamformer can improve the response in a selected direction. The performance of the beamformer depends on parameters such as the number of hydrophones, spacing between elements, signal frequency, propagation speed, and the direction of arrival.

Beamforming can be studied using one-dimensional (1D) and two-dimensional (2D) array configurations. A 1D beamformer generally uses a linear array of hydrophones arranged along a straight line. Appropriate time delays are applied to the received signals so that they add constructively in the desired direction. A 2D beamformer uses a planar array, in which hydrophones are arranged along two dimensions. This configuration provides control over the directional response in both azimuth and elevation. The array factor and normalized beam pattern are useful for understanding the main lobe, side lobes, and grating lobes of the array.

MATLAB is a useful tool for analyzing beamforming systems because it supports mathematical computation, signal generation, visualization, and simulation. It can be used to generate test signals, calculate propagation delays, align received signals, calculate array factors, and plot beam patterns. These functions help evaluate the performance of beamforming algorithms before developing a hardware-oriented implementation.

Xilinx Vivado is used for digital hardware design and simulation. It supports Verilog Hardware Description Language (HDL), Register Transfer Level (RTL) design, IP core integration, memory initialization, and waveform-based verification. In this internship, Vivado 2019.1 was used to study digital signal processing blocks and verify their behavior through simulation. The DDS Compiler IP was used for sine wave generation, and clock-cycle delays were studied using RTL-based designs. COE and MEM files were used where required to initialize memory with filter coefficients and input samples.

The internship also included the generation and analysis of Linear Frequency Modulated (LFM) signals and the design of a 10× polyphase interpolation system

using a 60th-order FIR filter. These activities provided a broader understanding of signal generation, filtering, sampling-rate conversion, and the transfer of MATLAB-generated data into a hardware-oriented design environment.

The main focus of this work is the study and implementation of 1D linear time-delay beamforming and the MATLAB-based analysis of 2D planar array beamforming. The 1D beamformer demonstrates how time delays can align signals received by a linear hydrophone array, while the planar array analysis extends the concept to two-dimensional geometry and directional beam patterns. MATLAB was used for algorithm development and performance analysis, while Vivado was used for RTL design and simulation of the relevant digital processing blocks.

This internship helped develop practical knowledge of digital signal processing and FPGA-oriented design methodologies. It also provided an understanding of how mathematical models can be converted into simulation-based digital systems. The work establishes a foundation for future development, optimization, and possible hardware implementation of beamforming systems for underwater acoustic and other sensor array applications.

2. OBJECTIVES OF THE INTERNSHIP

The main objective of the internship was to gain practical knowledge of digital signal processing and its implementation using MATLAB and Xilinx Vivado. The specific objectives were:

1.To understand the Xilinx Vivado design environment, including project creation, IP integration, memory initialization, Verilog HDL coding, and simulation.

2.To generate sine waves using the DDS Compiler IP and study the effects of different clock-cycle delays on the output.

3.To generate LFM signals using MATLAB and analyze their time-domain and frequency characteristics.

4.To develop a linear time-delay beamformer in MATLAB and transfer the generated data to Vivado for simulation.

5.To design and simulate a 10× polyphase interpolation system using a 60th-order FIR filter.

6.To understand the use of COE and MEM files for transferring filter coefficients and input samples into the Vivado design.

7.To develop a planar array beamformer in MATLAB and observes the elevation cut and azimuthal cut.

CHAPTER 1: SOFTWARE TOOLS AND TECHNOLOGIES

1.1 MATLAB 2021

MATLAB 2021 is a numerical computing software used for mathematical calculations, signal processing, algorithm development, and data analysis. It provides functions for generating signals, designing digital filters, and analyzing waveforms.

1.2 Xilinx Vivado 2019.1

Xilinx Vivado 2019.1 is a design tool used to develop and simulate digital circuits for compatible Xilinx FPGA devices. It supports RTL coding, IP integration, memory configuration, and waveform analysis.

Its main applications during the internship were:

> Creating projects and integrating IP cores.

> Implementing Verilog HDL designs.

> Configuring memory blocks and initialization files.

> Simulating sine wave generation, delays, and signal processing operations.

> All designs described in this report were verified through simulation rather than physical FPGA testing.

9. 1.3 Verilog Hardware Description Language

Verilog HDL is used to describe the behavior and structure of digital circuits. It supports combinational logic, sequential circuits, registers, counters, and memory interfaces.

During the internship, Verilog HDL was used to implement clock-cycle delays and processing logic for memory-based designs.

Important concepts include:

Clock: Controls the timing of sequential operations.

Register: Stores digital data between clock events.

Reset: Initializes a circuit to a known state.

RTL design: Describes data transfers and logical operations at the register-transfer level.

1.4 DDS Compiler IP

The Direct Digital Synthesis (DDS) Compiler IP is a configurable Vivado component used to generate digital waveforms, such as sine and cosine signals.

DDS generates a waveform by updating a digital phase accumulator and converting the phase information into amplitude values.

The output frequency is related to the phase increment by:

$$f_out = \frac{\Delta\Phi}{2^P} f_{clk}$$

Where $(f_{out}) = output frequency$.

$(\Delta\Phi) =$ ¿ phase increment.

P= phase accumulator width in bits.
Q= input clock frequency.

DDS Compiler IP was used to generate a sine wave, which was verified through Vivado simulation.

1.5 Block Memory Generator IP

The Block Memory Generator IP provides configurable memory functionality for storing and accessing digital data in compatible FPGA designs.

In the internship, memory blocks were used to store input samples and filter coefficients for digital signal processing operations.

Important memory parameters include:

Data width: Number of bits stored per location.

Memory depth: Number of available locations.

Address width: Number of bits required to address the locations.

Initialization: Loading predefined values into memory.

The total memory capacity is:

$C = D \times W$

Where (C) is capacity in bits, (D) is memory depth, and (W) is data width in bits.

1.6 COE and MEM Files

>COE and MEM files are used to provide predefined data to memory structures or compatible design workflows.

>A COE (Coefficient) file is supported by certain Xilinx IP configurations for specifying memory initialization values.

> COE files were used to transfer MATLAB-generated filter coefficients and required beamformer data into Vivado.

>A MEM file provides memory data in a format supported by the selected tool and loading method.

During the interpolation task, input samples were stored in a MEM file and added to the Vivado project.

1.7 RTL Design and Simulation

Register Transfer Level (RTL) design describes digital circuit behavior in terms of registers, data transfers, and logical operations. Verilog HDL is commonly used to implement RTL designs.Simulation checks the logical behavior of a design over time without requiring physical hardware. The simulation workflow includes:

1,Creating the project and adding source files.
2.Integrating the required IP cores.
3.Writing Verilog RTL code.
4.Configuring memory initialization.
5.Running the simulation.
6.Examining waveforms and verifying expected behavior.

1.8 MATLAB-to-Vivado Integration

MATLAB and Vivado were used together to develop and simulate digital signal processing systems. MATLAB handled mathematical operations, signal generation, and filter design, while Vivado handled RTL implementation, memory integration, and simulation.The general workflow was:

1.Generate signals or calculate filter coefficients in MATLAB.
2.Export the required data into compatible COE or MEM files.
3.Add the files to the Vivado project.
4.Integrate the memory blocks and Verilog RTL.
5.Run simulation and examine the output waveforms.

CHAPTER 2: SINE WAVE GENERATION USING DDS COMPILER

2.1 Introduction

A sine wave is a periodic signal widely used in digital signal processing and communication systems. In this task, a sine wave was generated using the DDS Compiler IP in Xilinx Vivado 2019.1 and verified through simulation.

2.2 Principle of Direct Digital Synthesis

Direct Digital Synthesis (DDS) is a digital technique used to generate periodic signals. It uses phase accumulation and phase-to-amplitude conversion to produce digital sine wave samples.

The output frequency is given by: $f_{out} = \frac{\Delta\Phi}{2^P} f_{clk}$

Where (f_{out} is the output frequency, $(\Delta\Phi)$ is the phase increment, (P) is the phase accumulator width, and) is the input clock frequency.

2.3 DDS Compiler IP Configuration

The DDS Compiler IP is a predefined Vivado component used to generate digital waveforms. It was configured for sine wave generation based on the required design settings.

Important parameters include clock frequency, phase accumulator width, phase increment, and output data width.

2.4 Design Methodology

The following steps were performed:

- Created a project in Vivado 2019.1.
- Selected and configured the DDS Compiler IP.
- Connected the required clock and control signals.
- Generated the IP output products.
- Ran the simulation and observed the output waveform.

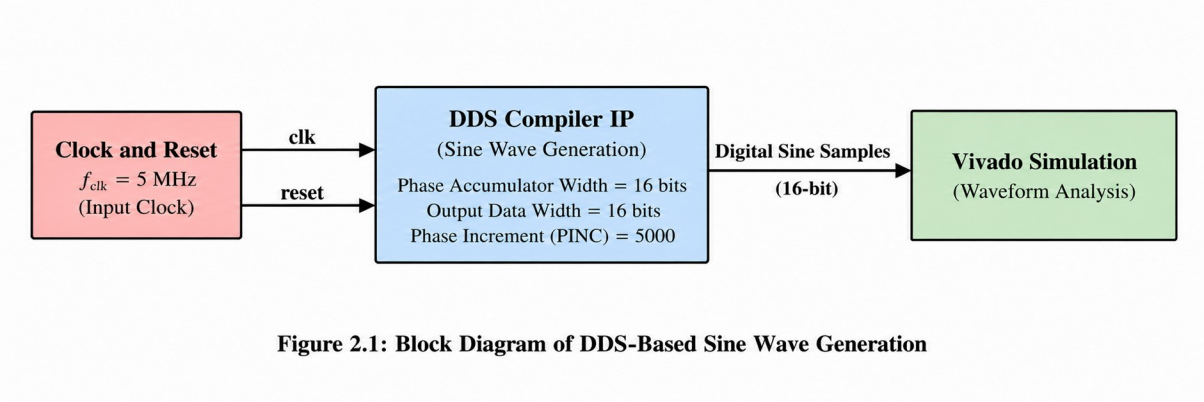


Figure 2.1: Block Diagram of DDS-Based Sine Wave Generation

2.5 DDS Compiler Configuration and Parameter Calculation

The DDS Compiler IP in Xilinx Vivado 2019.1 was configured to generate a digital sine wave using a system clock of 5 MHz and a 16-bit phase accumulator. The phase increment was selected to obtain the required output frequency.

Parameter	Value
System clock frequency (f_clk)	5 MHz
Desired output frequency (f_out)	Approximately 381.47 kHz
Phase accumulator width ((N))	16 bits
Output data width	16 bits
Phase increment (PINC)	5000
Frequency resolution	Approximately 76.294 Hz

The frequency resolution of a DDS system depends on the system clock frequency and the phase accumulator width.

$$\Delta f = \frac{f_{clk}}{2^N}$$

Substituting the values: $\backslash Deltaf = \frac{5 \times 10^6}{2^{16}} \Delta f \approx 76.294\, Hz$

Therefore, the frequency resolution is approximately 76.294 Hz.

The DDS output frequency is calculated using:

$$f_{out} = \frac{PINC \times f_{clk}}{2^N}$$

Rearranging the equation to calculate the phase increment:

$$PINC = \frac{f_{out} \times 2^N}{f_{clk}}$$

For a phase increment of 5000: $f_{out} = \frac{5000 \times 5 \times 10^6}{2^{16}}$

$f_{out} \approx 381.47\ kHz$

Thus, a phase increment of 5000 produces an output frequency of approximately 381.47 kHz for a 5 MHz clock and a 16-bit phase accumulator.

The phase increment must be entered in the format required by the DDS Compiler configuration. The decimal value 5000 is represented in binary as:

$5000_{10} = 0001,0011,1000,1000_2$

This is the 16-bit binary representation of the phase increment.The approximate number of system-clock cycles per output sine wave cycle is:

$$N_s = \frac{f_{clk}}{f_{out}}$$

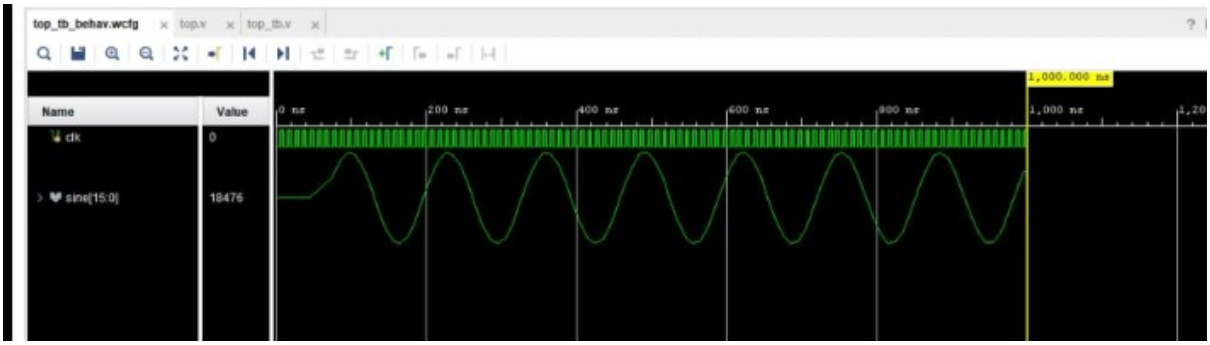
$$N_s = \frac{5 \times 10^6}{381.47 \times 10^3}$$

$N_s \approx 13.11\ clock cycles$

This means that one output sine wave cycle spans approximately 13.11 system-clock cycles. The number is not an integer because the DDS phase accumulator advances by a fixed phase increment at every clock cycle.

2.6 Simulation Results and Waveform Analysis

Results:- Sine wave is generated using DDS compiler in vivado



CHAPTER 3: DELAYED SINE WAVE GENERATION

3.1 Introduction

In this task, different clock-cycle delays were applied to the sine wave generated using the DDS Compiler IP in Xilinx Vivado 2019.1. The delayed outputs were verified through simulation to understand the effect of clock-cycle delays on signal timing.

3.2 Concept of Clock-Cycle Delay

A clock-cycle delay means postponing a digital signal by a specified number of clock cycles. It can be implemented using registers connected in sequence.

For a clock period (Tclk), the delay introduced by (N) clock cycles is:

$T_{delay} = N\,T_{clk}\, Where:$

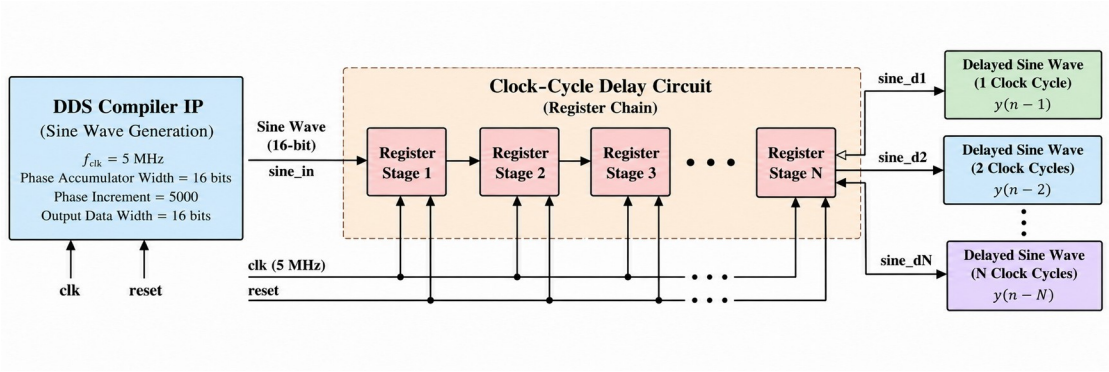
$\left(T_{delay}\right) = Totaldelaytime.$

$\mathrm{N} = Numberofclockcycles.$

$\left(T_{clk}\right) = Clockperiod.$

3.3 Design Methodology

The DDS Compiler IP generated the sine wave, which was then passed through delay logic implemented using Verilog HDL. Different clock-cycle delays were applied, and the corresponding outputs were observed in Vivado simulation.



3.4 Verilog RTL Implementation

The delay circuit can be implemented using clocked registers. Each register stores the input value and transfers it to the next stage at a clock edge.

```
 always @(posedge clk) begin
    delay1 <= sine_in;
    delay2 <= delay1;
    delay3 <= delay2;
end
```

Here, `clk` is the clock signal, `sine_in` is the input sample, and `delay1`, `delay2`, and `delay3` are registers that provide successive clock-cycle delays. This is an illustrative example; the actual implementation depends on the delay stages used in the project.

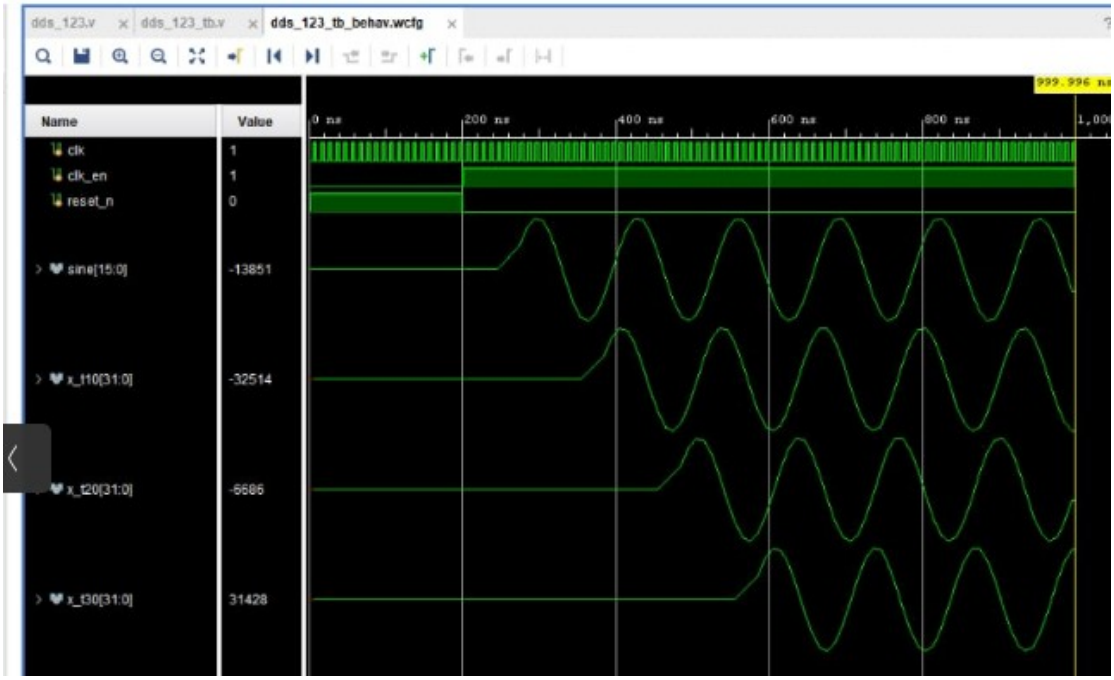
3.5 Simulation Setup

The design was simulated in Xilinx Vivado 2019.1. The original sine wave and delayed outputs were observed using the waveform viewer to check the timing difference between the signals.

10.3.6 Output Waveform Analysis

The simulation waveforms were used to compare the original sine wave with the delayed outputs. Increasing the number of register stages increases the delay in clock cycles while preserving the sequence of input samples.

Figure 3.2: Original and Delayed Sine Waveforms



Result: Different clock-cycle delays were applied to the DDS-generated sine wave, and the delayed outputs were verified through Vivado simulation.

CHAPTER 4: LFM SIGNAL GENERATION USING MATLAB

4.1 Introduction

Linear Frequency Modulation (LFM) is an important signal modulation technique used in radar, communication, and digital signal processing applications. In an LFM signal, the instantaneous frequency varies linearly with time while the signal amplitude generally remains constant. Because of this characteristic, an LFM signal is also known as a chirp signal.

LFM signals are particularly useful in radar systems because they provide good range resolution and allow the transmitted signal energy to be distributed over a relatively long pulse duration. They are also useful for studying frequency variation, signal bandwidth, and time-frequency characteristics.

4.2 Principle of Linear Frequency Modulation

In Linear Frequency Modulation, the instantaneous frequency changes at a constant rate with respect to time. Depending on the direction of the frequency sweep, the signal can be classified as an upward chirp or a downward chirp.

Upward chirp: The instantaneous frequency increases linearly with time.

Downward chirp: The instantaneous frequency decreases linearly with time.

The frequency sweep is controlled by the starting frequency, ending frequency, and signal duration. The rate of frequency change is called the frequency sweep rate or chirp rate.

For an LFM signal, the instantaneous frequency is expressed as:

$$f(t) = f_0 + kt$$

Here,) is the instantaneous frequency at time (t), (f_0) is the starting frequency, and (k) is the frequency sweep rate.

If the signal sweeps from$(f_0)(f_1)$ over a duration (T), the chirp rate is:

$$k = \frac{f_1 - f_0}{T}$$

A positive value of (k) represents an upward frequency sweep, while a negative value represents a downward frequency sweep.

The principle of LFM is based on changing the instantaneous phase so that its derivative produces the required linear frequency variation. This relationship forms the basis for generating an LFM waveform mathematically in MATLAB.

4.3 Mathematical Representation of LFM Signal

A real-valued LFM signal can be represented as:

$$s(t) = A\cos\left[2\pi\left(f_0 t + \frac{k}{2}t^2\right) + \phi_0\right]$$

Where:(s(t)) = Instantaneous signal amplitude at time (t).

A= Signal amplitude.

(t) = Time in seconds.

(f_0) = Starting frequency in hertz.

(k) = Frequency sweep rate in hertz per second.

(ϕ_0) = Initial phase in radians..

The instantaneous frequency is obtained by differentiating the total phase with respect to time and dividing by (2\pi):

$$f(t) = \frac{1}{2\pi}\frac{d\phi(t)}{dt}$$

where $(\phi(t))$ is the total instantaneous phase in radians. Substituting the phase expression gives: f(t)=f_0+kt.

The bandwidth of the LFM signal is:

$B = f_1 - f_0$ ∨

Where (B) is the frequency sweep bandwidth in hertz. A larger frequency sweep bandwidth indicates a wider range of frequencies within the signal.

4.4 MATLAB Implementation

MATLAB 2021 was used to generate the LFM signal by defining the sampling frequency, signal duration, starting frequency, and ending frequency. These parameters determine the time samples and the frequency sweep of the generated waveform.

The general implementation procedure consists of the following steps:

1.Define the sampling frequency and signal duration.
2.Specify the starting and ending frequencies.
3.Calculate the chirp rate using the frequency difference and duration.
4.Generate a time vector with suitable sampling intervals.
5.Evaluate the LFM signal equation at each time sample.
6.Plot the generated signal and analyze its characteristics.

4.5 Time-Domain Analysis

The time-domain representation shows how the signal amplitude changes with time. An LFM signal appears as a sequence of oscillations whose spacing changes as the instantaneous frequency varies.

For an upward chirp, the frequency increases over time. Consequently, the oscillations become closer together as time progresses. For a downward chirp, the frequency decreases, and the oscillations become more widely spaced.

The time-domain plot is useful for examining the signal duration, amplitude, continuity, and general waveform behavior. However, a time-domain plot alone does not clearly show the complete frequency variation. Therefore, it can be supplemented with an instantaneous-frequency plot or a time-frequency representation.

4.6 Frequency Characteristics

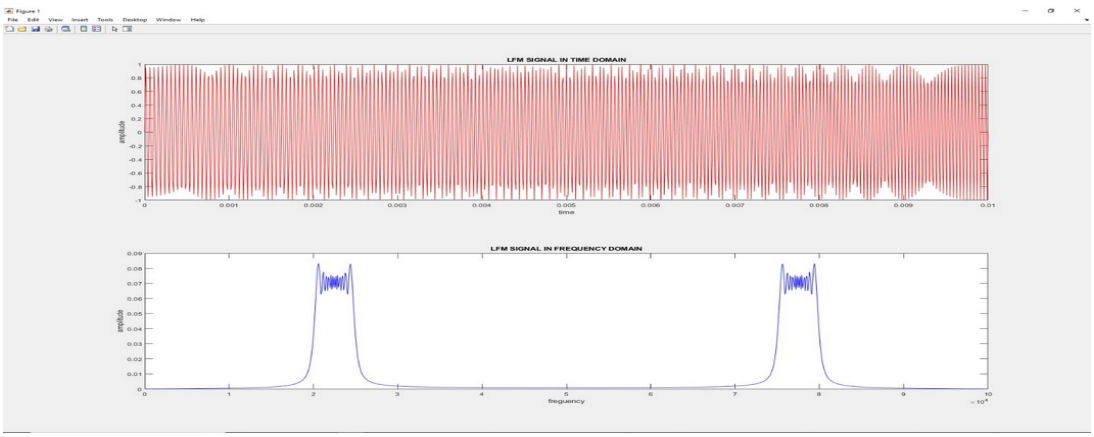
The frequency characteristics of an LFM signal describe how its frequency changes during the signal duration. For an ideal linear sweep, the instantaneous frequency follows a straight line when plotted against time.

The relationship is: $f(t)=f_0+kt$

A spectrogram can also be used to visualize the frequency content over time. In an LFM spectrogram, the energy typically follows a sloping line across the time-frequency plane, showing the direction and rate of the frequency sweep.

The frequency plots should be interpreted according to the actual starting frequency, ending frequency, and signal duration. A spectrogram should only be included if it was generated as part of the analysis.

4.7 Results and Discussion



Result: An LFM signal was generated and analyzed using MATLAB. Its time-domain behavior and linear variation of instantaneous frequency were studied using the generated plots.

CHAPTER 5: LINEAR TIME-DELAY BEAMFORMER

5.1 Introduction

Beamforming is a signal processing technique used to enhance signals arriving from a particular direction. In underwater applications, hydrophones are arranged in an array to receive acoustic signals travelling through water. By applying suitable time delays and combining the received signals, the system can improve its response to signals from the desired direction.

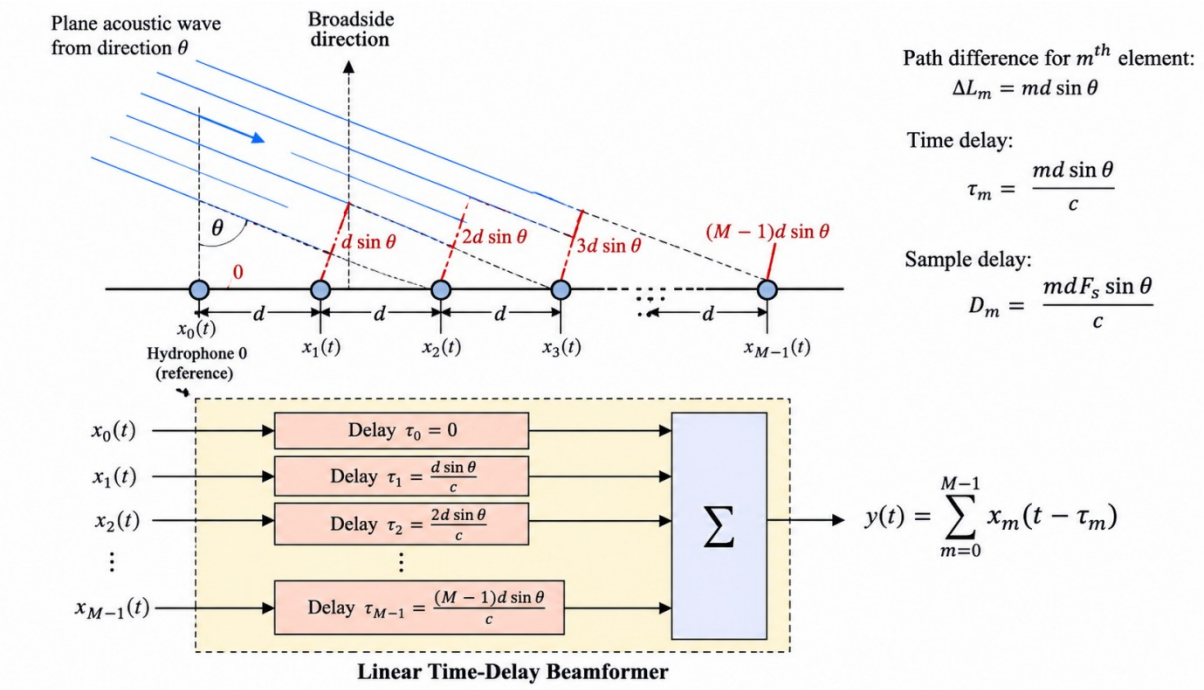
5.2 Basic Principle of Beamforming

When an underwater acoustic wave reaches different hydrophones, it arrives at slightly different times. Beamforming compensates for these time differences by delaying the signals before combining them.

The main steps are:

1. Receive acoustic signals through hydrophones.
2. Apply appropriate time delays.
3. Combine the delayed signals.
4. Observe the resulting output.

Figure 5.1: Basic Block Diagram of an Underwater Beamformer



5.3 Linear Time-Delay Beamforming Concept

For an underwater acoustic system, a linear time-delay beamformer uses a linear array of hydrophones to receive sound waves travelling through water. Since the sound wave reaches each hydrophone at a different time, the propagation delays are calculated and compensated for to align the received signals.

Consider a linear array consisting of (M) hydrophones separated by an equal distance (d). A plane acoustic wave arrives at an angle (theta), measured from the **broadside direction** of the array.

Let:

- (M) = Number of hydrophones.
- (d) = Spacing between adjacent hydrophones (m).
- (theta) = Angle of arrival measured from broadside.
- (c) = Speed of sound in water (m/s).
- (m) = Hydrophone index, where (m=0,1,…,M-1).

The first hydrophone is selected as the reference point.

For a wave arriving at an angle (theta) from broadside, the additional propagation distance between adjacent hydrophones is obtained by projecting their separation onto the direction of propagation.

From the geometry of the array,

$$\sin\theta = \frac{Path difference}{d}$$

Therefore, the path difference between adjacent hydrophones is

$$\Delta L = d\sin\theta$$

For the $(m)-$th hydrophone, $\Delta L_m = md\sin\theta$

where (ΔL_m) is the relative propagation path difference for hydrophone (m).

The propagation time is equal to the distance travelled divided by the speed of sound.

$$Time = \frac{Distance}{Speed}$$

Substituting the path difference,

$$\tau_m = \frac{\Delta L_m}{c}$$

Since $(\Delta L_m = md\sin\theta)$,

$$\tau_m = \frac{md\sin\theta}{c}$$

This is the propagation time difference relative to the first hydrophone.

For the reference hydrophone, (m=0): $\tau_0 = 0$

For the second hydrophone, (m=1):$\tau_1 = \frac{d \sin\theta}{c}$

For the third hydrophone, (m=2): $tau_2 = \frac{2d \sin\theta}{c}$

Therefore, the delay increases linearly with the hydrophone index.

In a digital beamformer, the received acoustic signals are represented as discrete-time samples. Therefore, the calculated time delay must be converted into a number of samples.

Let (F_s)be the sampling frequency. The sampling period is $T_s = \frac{1}{F_s}$

The sample delay for the (m)-th hydrophone is

$D_m = \frac{\tau_m}{T_s}$

Since$(T_s = 1/F_s)$,

$D_m = \tau_m F_s$

Substituting the time-delay equation gives

$D_m = \frac{md F_s \sin\theta}{c}$

where (D_m) is the delay expressed in samples.

After compensating for the propagation delays, the received signals are aligned and combined to produce the beamformer output.

The output is expressed as

$$y(t) = \sum_{m=0}^{M-1} x_m(t - \tau_m)$$

where:

- $(y(t))$ = Combined beamformer output.
- $(x_m(t))$= Signal received by hydrophone \(m\).
- (τ_m) = Applied delay for hydrophone \(m\).
- (M) = Total number of hydrophones.

For a digital implementation, the corresponding expression is

$$y[n] = \sum_{m=0}^{M-1} x_m[n - D_m]$$

The main equation for the propagation time delay of a linear time-delay beamformer

$$\tau_m = \frac{md \sin\theta}{c}$$

5.4 Grating Lobes Concept

Grating lobes are unwanted secondary peaks that appear in the directional response of a sensor array when the spacing between its elements is too large for the operating wavelength and steering angle.

- (λ) = Acoustic wavelength in metres.
- (c_w)= Speed of sound in water in metres per second.
- (f) = Acoustic signal frequency in hertz.

To avoid grating lobes across the full visible angular range in a conventional uniformly spaced linear array, a common spacing condition is:

$$d \leq \frac{\lambda_{min}}{2}$$

Since the wavelength decreases as frequency increases, the minimum wavelength corresponds to the highest frequency of interest, (f_{max}):

$$\lambda_{min} = \frac{c_w}{f_{max}}$$

Therefore, the maximum recommended sensor spacing is:

$$d \leq \frac{c_w}{2 f_{max}}$$

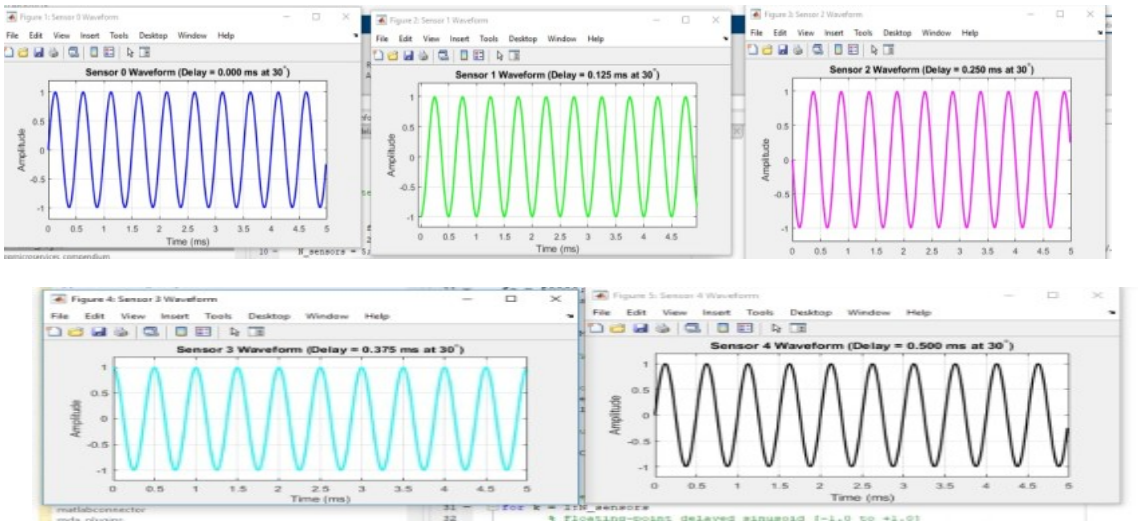
This condition helps prevent spatial aliasing and unwanted grating lobes across the full visible angular range.

Effect of Sensor Spacing

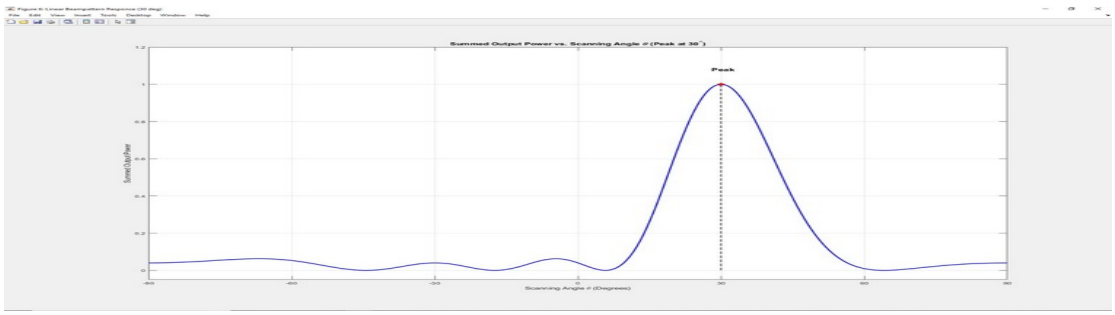
- **Spacing greater than half the minimum wavelength:** Grating lobes may appear, depending on the steering angle.
- **Spacing equal to half the minimum wavelength:** A common design choice that helps avoid grating lobes across the full visible angular range.
- **Spacing smaller than half the minimum wavelength:** Helps avoid grating lobes, but for a fixed number of hydrophones, it reduces the total array aperture and can produce a wider main beam.

5.5 MATLAB Design and Implementation

MATLAB 2021 was used to develop the linear time-delay beamformer and generate the required data . These are delayed waveforms of sensor 0,1,2,3,4



linear time delay beamformer in matlab



5.6 Data Export Using COE Files

The required data generated in MATLAB was exported using COE files for integration into Vivado. These files contain initialization values that can be loaded into compatible memory IP.

The data was formatted appropriately and added to the Vivado project. Correct data representation and memory configuration are important for obtaining the expected simulation behavior.

5.7 Vivado Integration and Simulation

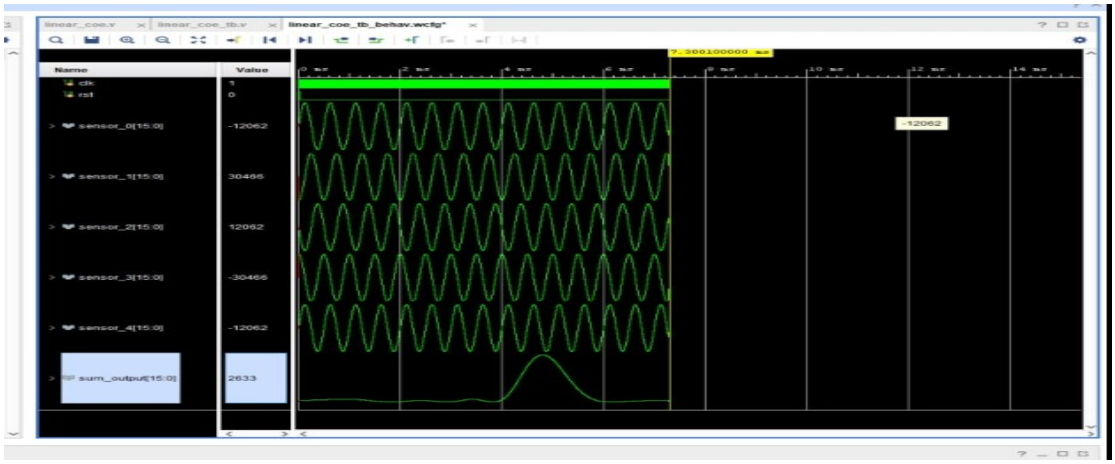
Xilinx Vivado 2019.1 was used to integrate the MATLAB-generated data into the design environment. The COE files were used to initialize compatible memory blocks, and the design was prepared for simulation.

The output waveforms were examined to study the behavior of the design under the applied simulation conditions.

5.8 Results and Discussion

The linear time-delay beamformer was developed using MATLAB, and the required data was transferred to Vivado through COE files for simulation. This work improved my understanding of underwater acoustic beamforming, time-delay calculations, hydrophone spacing, and digital design integration.

Result: The beamformer data was generated in MATLAB, imported into Vivado, and used in the simulation workflow.



CHAPTER 6: 10× POLYPHASE INTERPOLATION USING A 60TH-ORDER FIR FILTER

6.1 Introduction

Interpolation is a digital signal processing technique used to increase the sampling frequency of a discrete-time signal ,while maintaining its original signal information as accurately as possible.

In digital signal processing, some systems require a higher sampling rate than the original input provides. Instead of acquiring the signal again at a higher rate, interpolation allows us to generate additional samples digitally.In this project, a 2 kHz input signal sampled at 100 kHz was processed using a 10× polyphase interpolation system. The output sampling frequency was increased to 1 MHz.

A 60th-order Finite Impulse Response (FIR) low-pass filter was designed using MATLAB 2021. The filter coefficients were exported to a COE file, while the input samples were stored in a MEM file. Verilog HDL and memory blocks were used in Xilinx Vivado 2019.1 to implement and simulate the processing system.

6.2 Fundamentals of Interpolation

Interpolation increases the sampling rate by inserting additional samples between the original input samples. For an interpolation factor (L), the output sampling frequency is:

$F_{s,out} = L F_{s,\in}$

Where: $\left(F_{s,out}\right) = Outputsamplingfrequency$.

(L) = Interpolation factor.
$F_{s,out} = 10 \times 100, kHz$

The input signal frequency was 2 kHz. Increasing the sampling frequency provides more samples to represent the signal digitally; it does not change the original signal frequency.

6.3 Polyphase Interpolation Principle

Polyphase interpolation is an efficient implementation technique that divides an FIR filter into multiple smaller subfilters called polyphase branches. Instead of calculating every intermediate high-rate filtered sample, the polyphase structure processes the required samples using the individual branches.

The interpolated output can be expressed as:

$$y[n]=\sum_{k=0}^{N} h[k]\,x_{\uparrow L}[n-k]$$

Where:(y[n]) = Output sample.

(h[k]) = FIR filter coefficient.

$(x_{\uparrow L}[n])$= Input sequence after upsampling by (L).

(N) = FIR filter order.

(L) = Interpolation factor.

For a 10× interpolation system, the filter is divided into 10 polyphase branches. Since the filter order is 60, the total number of coefficients is:

N+1=60+1=61

6.4 FIR Filter Design Using MATLAB

MATLAB 2021 was used to design the 60th-order FIR low-pass filter. The filter removes unwanted high-frequency components introduced by the upsampling process and helps preserve the desired signal.

The filter parameters used in the project were:

Parameter	**Value**
Input sampling frequency	100 kHz
Output sampling frequency	MHz
Input signal frequency	2 kHz
Interpolation factor	10
FIR filter order	60
Number of coefficients	61
Coefficient format	16-bit signed Q15
Number of polyphase branches	10

The filter cutoff frequency should be selected according to the required passband and transition band. The actual cutoff frequency used in the MATLAB design should be recorded from the implemented code.

The filter coefficients were converted into a 16-bit signed Q15 representation for use in the digital design. Q15 represents fixed-point values using a 16-bit signed word, with 15 fractional bits.

The approximate conversion from a normalized floating-point coefficient (h[k]) to an integer coefficient is:

$$H[k] = round(h[k]2^{15})$$

Where (H[k]) is the quantized integer coefficient and (h[k]) is the original floating-point coefficient. The quantized values must fit within the signed 16-bit range.

6.5 10× Interpolation Architecture

The system increases the sampling rate by a factor of 10 and uses the FIR filter to reconstruct the interpolated sequence.

Block Diagram

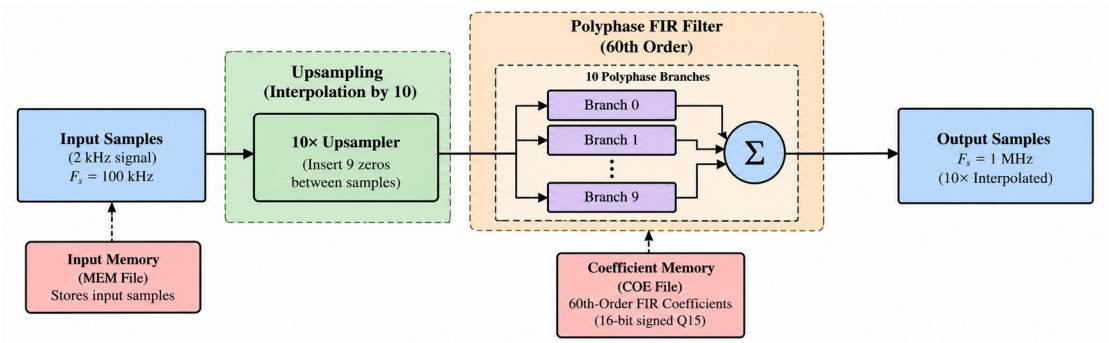


Figure 6.1: Block Diagram of 10× Polyphase Interpolation

after generating it in the matlab we can export input samples as memory file and coefficients as .coe files into the vivado we can get simulated waveform in the vivado

6.6 Exporting Filter Coefficients Using COE Files

After designing the FIR filter in MATLAB, the coefficients were converted to the required fixed-point representation and exported to a COE file.

The procedure involved:

1. Designing the FIR filter in MATLAB.

2. Quantizing the coefficients to signed Q15 format.

3. Converting the coefficients into the required memory representation.

4. Saving the coefficient values in a COE file.

5. Configuring the Vivado memory IP to use the initialization file.

6. Correct coefficient ordering, radix, data width, and signed-number representation are essential for accurate processing.

6.7 Input Sample Initialization Using MEM Files

The input samples were stored in a MEM file and added to the Vivado project. The memory was configured to store the samples in the required digital format.

During simulation, the Verilog design accessed the stored input samples and supplied them to the processing logic.

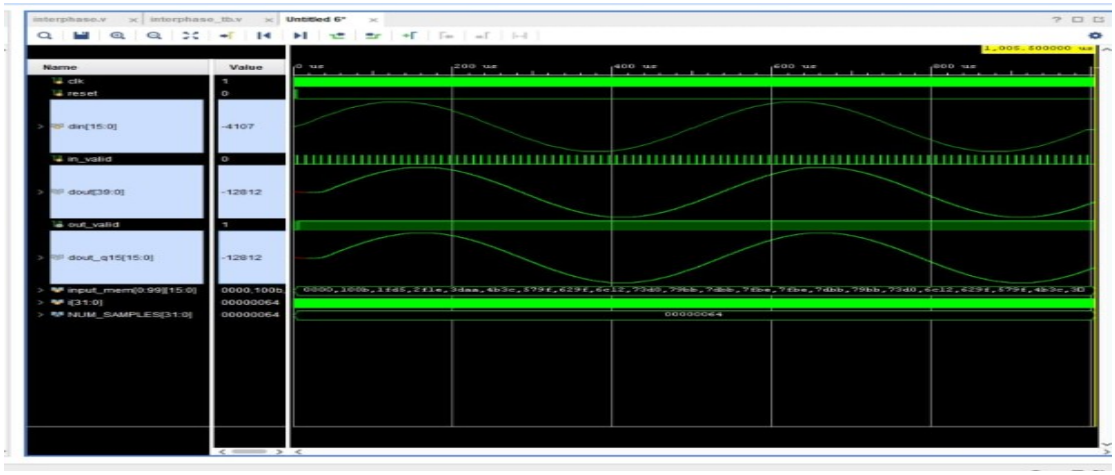
The COE file and MEM file served different purposes: the COE file stored the filter coefficients, while the MEM file stored the input samples.

6.8 Vivado Simulation and Waveform Analysis

The interpolation design was simulated using Xilinx Vivado 2019.1. The input samples and filter coefficients were loaded from their respective initialization files.

The simulation waveforms were used to examine the processing sequence and output behavior. The expected output sampling rate for a 10× interpolation factor was 1 MHz, compared with the original 100 kHz input sampling rate.

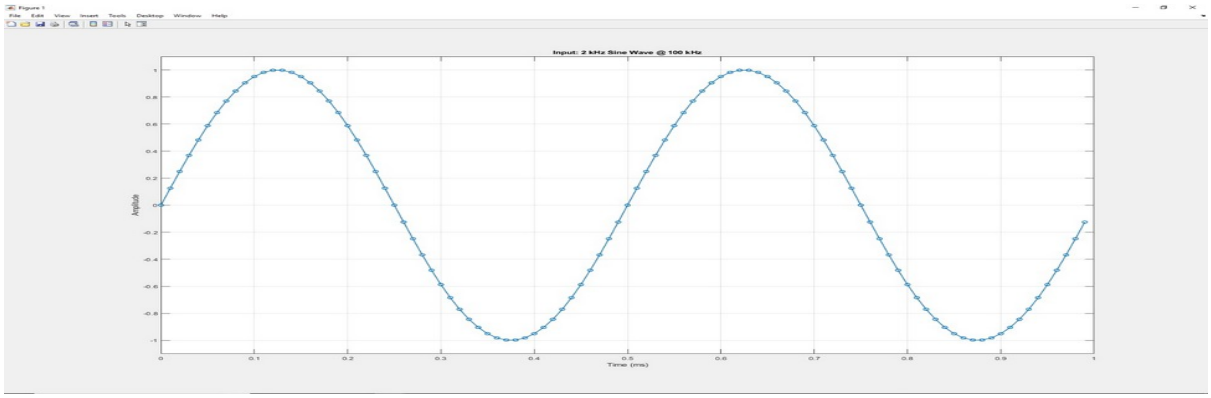
Figure 6.2: Vivado Simulation Waveform of the Interpolation System



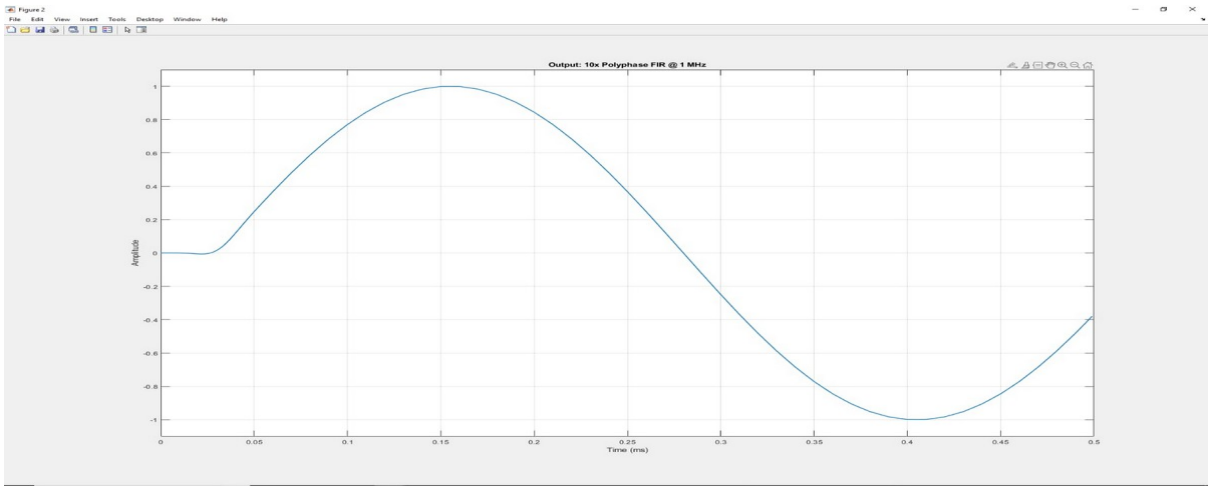
The simulation results should be checked for correct memory access, coefficient processing, output timing, and sample sequence. Actual waveform observations should be described using the results obtained from the project.

6.9 Results and Discussion

Result: The 10× interpolation design was developed and simulated with a 100 kHz input sampling frequency and a target output sampling frequency of 1 MHz.



After applying the polyphase 10x interpolation we can get the signal sampled with 1Mhz



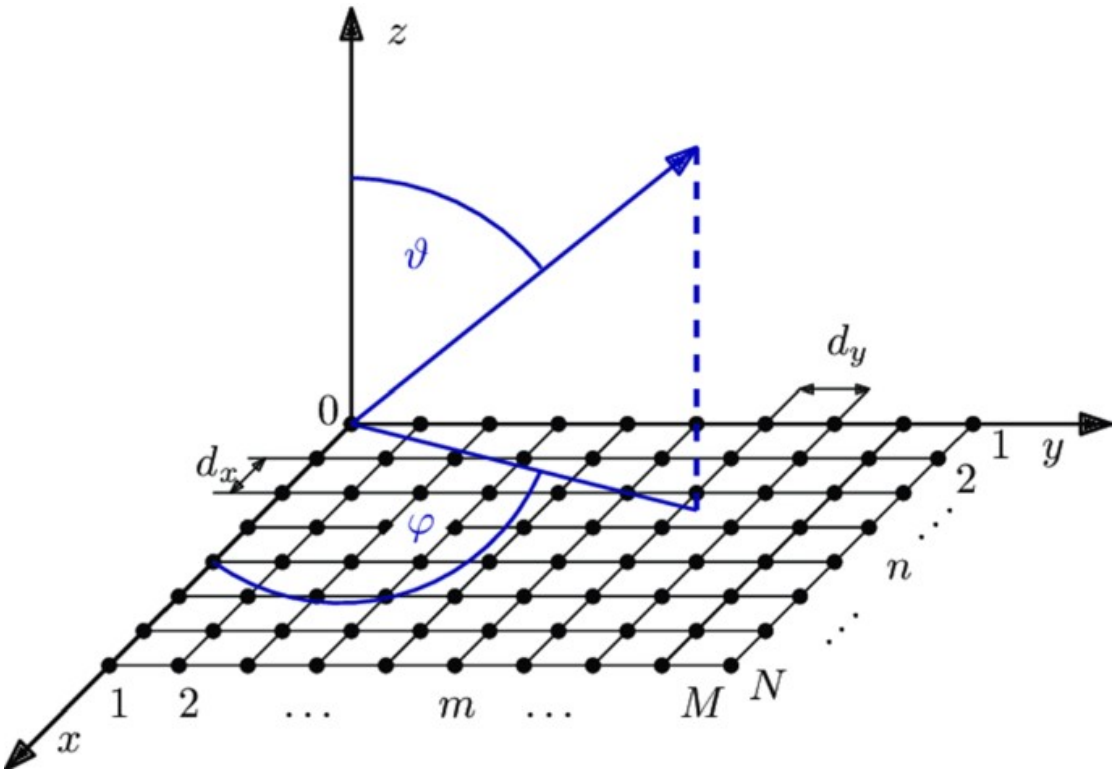
CHAPTER 7: PLANAR ARRAY BEAMFORMER

7.1 Introduction

A planar array beamformer uses multiple hydrophones arranged in a two-dimensional pattern to receive underwater sound signals. Since sound waves reach each hydrophone at different times, suitable time delays are applied to align the signals before combining them.

Unlike a linear array, a planar array can control the signal direction in two dimensions. This helps study signals arriving from different directions underwater..

7.2 Principle and Geometry of Planar Array Beamforming



Planar Array Beamformer / 2D Conventional Beamformer

7.3. Introduction

A planar array beamformer, also called a **2D conventional beamformer**, uses multiple hydrophones arranged in a two-dimensional pattern to receive underwater

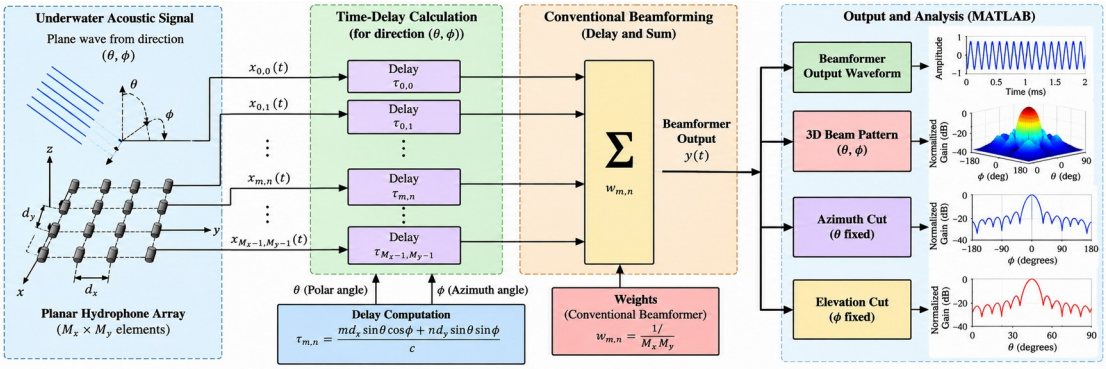
acoustic signals. The hydrophones are placed along two axes, usually the (x)-axis and (y)-axis.

When an underwater sound wave reaches the hydrophones, it arrives at different times because each hydrophone is located at a different position. The beamformer applies suitable time delays or phase shifts to align the received signals and then adds them together.

By controlling these delays, the beamformer can focus on signals arriving from a particular direction. Unlike a linear array, a planar array can analyze the signal direction in both **azimuth** ϕ and **elevation or polar angle** θ.

The planar array beamformer is used in underwater surveillance, sonar systems, acoustic signal processing, and direction-of-arrival estimation.

7.4 Planar Array Geometry



Consider a rectangular planar array containing \(M_x\) hydrophones along the x-axis and M_y hydrophones along the y-axis.

The total number of hydrophones is

$M = M_x \cdot M_y$

Let:

- d_x= spacing between hydrophones along the \(x\)-axis.
- d_y= spacing between hydrophones along the \(y\)-axis.
- M_x= number of array elements along the \(x\)-axis.
- M_y= number of array elements along the \(y\)-axis.
- c= speed of sound in water.
- lambda = wavelength of the received acoustic signal.

The position of the hydrophone at row (m) and column (n) is

$\boldsymbol{r}_{m,n} = \begin{bmatrix} md_x & nd_y & 0 \end{bmatrix}$

where

$m = 0, 1, \cdots, M_x - 1$

$n = 0, 1, \cdots, M_y - 1$

The array lies in the xy-plane.

7.5. Direction of Arrival

The direction of an incoming sound wave is described by two angles:

- **Azimuth angle** ϕ:-Specifies the direction around the array in the \(xy\)-plane.
- **Polar angle** θ**:** Measured from the normal to the array plane.

Using this angle convention, the unit vector pointing toward the incoming wave source is

$$\boldsymbol{u} = \begin{bmatrix} \sin\theta\cos\phi \\ \sin\theta\sin\phi \\ \cos\theta \end{bmatrix}$$

These angles allow the beamformer to analyze signals arriving from different directions in three-dimensional space.

7.6. Time-Delay Calculation

Because the hydrophones occupy different positions, the sound wave travels different distances to reach each element. Therefore, the received signals have different arrival times.

The path difference for the hydrophone at position \((m,n)\), relative to the reference element, is

$$\Delta L_{m,n} = m d_x \sin\theta\cos\phi + n d_y \sin\theta\sin\phi$$

The corresponding propagation time difference is

$$tau_{m,n} = \frac{DELTA\, L_{m,n}}{c}$$

Therefore,

$$rect\, tau_{m,n} = \frac{m d_x \sin\theta\cos\phi + n d_y \sin\theta\sin\phi}{c}$$

This equation calculates the relative time delay for each hydrophone for a given direction (θ, ϕ)

For a digital beamformer with sampling frequency F_s, the delay in samples is

$$rect\, D_{m,n} = tau_{m,n} F_s$$

In practical digital systems, the delay may need to be implemented using integer-sample delays or fractional-delay filters.

7.7. Conventional Beamforming Principle

Conventional beamforming aligns the signals received by the hydrophones for a selected direction and combines them.

Let $x_{m,n}(t)$ represent the signal received by the hydrophone at position (m,n). The beamformer output is

$$rect\ y(t) = \sum_{m=0}^{M_x-1} \sum_{n=0}^{M_y-1} w_{m,n}\, x_{m,n}(t + \tau_{m,n})$$

Here:

- \(y(t)\) is the beamformer output.
- $x_{m,n}(t)$ is the received signal.
- $w_{m,n}$ is the weight applied to each hydrophone.
- $\tau_{m,n}$is the relative delay calculated for the selected direction.

The plus sign assumes that \(\tau_{m,n}\) is defined as the propagation delay relative to the reference element. The required delay or advance depends on the signal and delay convention used in the implementation.

For a conventional beamformer, the weights are usually equal:

$$w_{m,n} = \frac{1}{M_x M_y}$$

Thus, the output becomes

$$rect\ y(t) = \frac{1}{M_x M_y} \sum_{m=0}^{M_x-1} \sum_{n=0}^{M_y-1} x_{m,n}(t + \tau_{m,n})$$

The normalization helps keep the output amplitude comparable as the number of hydrophones changes.

7.8 Array Factor Calculation

The array factor describes how the planar array responds to signals arriving from different directions. For a uniformly weighted rectangular array, the array factor is

$$rect\ AF(\theta, \phi) = \sum_{m=0}^{M_x-1} \sum_{n=0}^{M_y-1} e^{jk\,(m d_x \sin\theta\cos\phi + n d_y \sin\theta\sin\phi)}$$

where the wavenumber is

$$k = \frac{2\pi}{\lambda} = \frac{2\pi f}{c}$$

Here, (f) is the signal frequency.

For a beam steered toward the direction ¿$(\theta_0, \phi_0$¿)define

$$\psi_x = k\, d_x \left(\sin\theta\cos\phi - \sin\theta_0\cos\phi_0\right)$$

$$\psi_y = k\, d_y \left(\sin\theta\sin\phi - \sin\theta_0\sin\phi_0\right)$$

The steered array factor can then be expressed as

$$AF(\theta,\phi) = \sum_{m=0}^{M_x-1} e^{jm\psi_x} \sum_{n=0}^{M_y-1} e^{jn\psi_y}$$

This expression shows that the rectangular planar array response can be separated into contributions from the two array dimensions.

7.9. Normalized Beam Pattern

To compare the response at different angles, the array factor is normalized by its maximum magnitude:

$$AF_N(\theta,\phi) = ¿\, AF(\theta,\phi) \vee \frac{¿}{max¿}\, \theta,\phi \left| AF(\theta,\phi)\right|$$

The normalized beam pattern in decibels is

$$rect\ AF_{rmdB}(\theta,\phi) = 20\log_{10}\left(AF_N(\theta,\phi)\right)$$

The beam pattern helps identify the following characteristics:

- **Main lobe:** The direction of maximum response.
- **Side lobes:** Smaller responses in other directions.
- **Nulls:** Directions where the array response is very low.
- **Beamwidth:** The angular width of the main lobe.

7.10 Azimuth and Elevation Cuts

Since the planar array response depends on both \(\theta\) and \(\phi\), it can be represented as a two-dimensional angular pattern or examined using one-dimensional cuts.

Azimuth Cut

An azimuth cut plots the beam pattern against ϕ while keeping θ fixed.

$$AF_{rmaz}(\phi) = AF(\theta_{rm¿}, \phi)$$

This plot shows how the array responds to signals arriving from different azimuth directions at a selected polar angle.

Elevation Cut

An elevation or polar-angle cut plots the beam pattern against θ while keeping ϕ fixed.

$AF_{rm\,el}(\theta) = AF(\theta, \phi_{rm¿})$

This plot shows how the array response changes with polar angle for a selected azimuth direction.

Note: The exact meaning of an elevation cut depends on the angle convention used in the MATLAB code. State the fixed angle and its value when presenting each plot.

7.11. Hydrophone Spacing and Grating Lobes

Hydrophone spacing affects the beam pattern. If the spacing is too large relative to the wavelength, additional strong lobes called grating lobes may appear.

The wavelength is

$$\lambda = \frac{c}{f}$$

A common spacing guideline for reducing grating lobes over the full visible angular range is

$$d_x \leq \frac{\lambda_{min}}{2}$$

$$d_y \leq \frac{\lambda_{min}}{2}$$

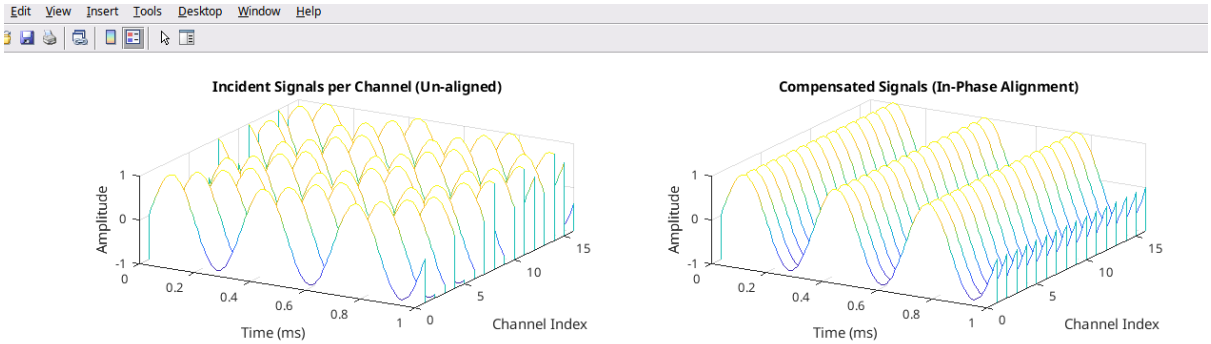
where $\lambda_{min} = \frac{c}{f_{max}}$ is the shortest wavelength in the operating frequency range.

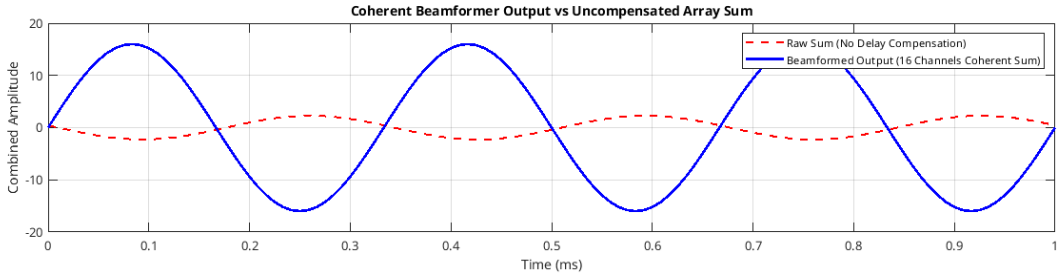
The actual spacing requirements depend on the operating frequency range, steering directions, and array geometry.

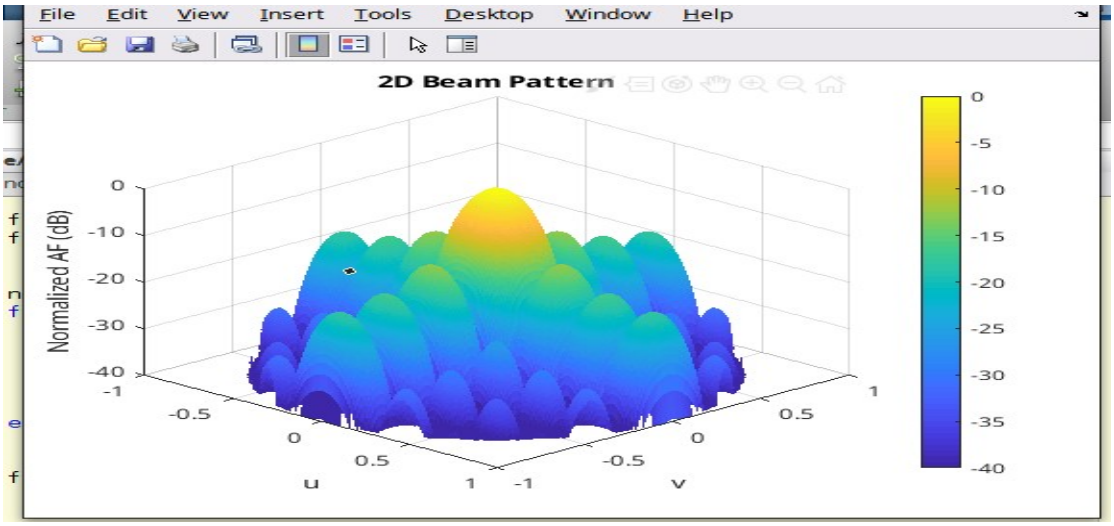
7.12. MATLAB Analysis

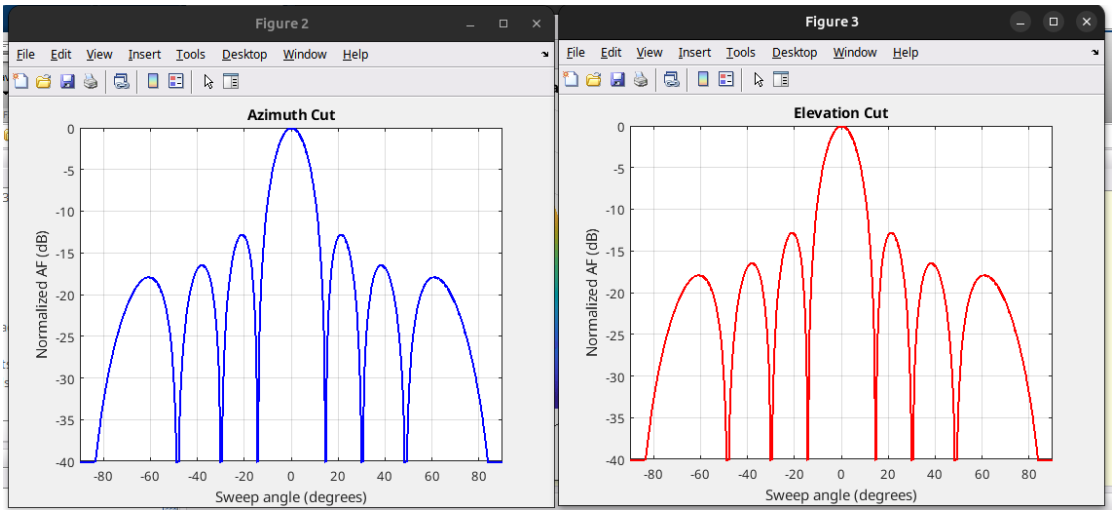
MATLAB can be used to calculate the time delays, evaluate the array factor, and plot the planar array beam pattern. The analysis can include:

1. Defining the number and spacing of hydrophones.
2. Setting the sound speed and operating frequency.
3. Selecting the steering direction using \(\theta_0\) and \(\phi_0\).
4. Calculating the relative delays for each element.
5. Computing the array factor over a range of angles.
6. Plotting the normalized beam pattern in decibels.
7. Generating azimuth and elevation cuts.


Edit View Insert Tools Desktop Window Help
Incident Signals per Channel (Un-aligned)
Amplitude
Time (ms)
Channel Index
Compensated Signals (In-Phase Alignment)
Amplitude
Time (ms)
Channel Index


Coherent Beamformer Output vs Uncompensated Array Sum
Raw Sum (No Delay Compensation)
Beamformed Output (16 Channels Coherent Sum)
Combined Amplitude
Time (ms)


File Edit View Insert Tools Desktop Window Help
2D Beam Pattern
Normalized AF (dB)
u
v


Figure 2
File Edit View Insert Tools Desktop Window Help
Azimuth Cut
Normalized AF (dB)
Sweep angle (degrees)
Figure 3
File Edit View Insert Tools Desktop Window Help
Elevation Cut
Normalized AF (dB)
Sweep angle (degrees)

CONCLUSION

During the internship at DRDO NSTL, I gained practical knowledge of digital signal processing, beamforming techniques, and FPGA-based design methodologies. MATLAB 2021 was used for signal generation, time-delay calculations, and beam pattern analysis, while Xilinx Vivado 2019.1 was used for Verilog RTL design, IP integration, and simulation. Sine wave generation using the DDS Compiler IP and delayed sine wave generation were studied and verified through simulation. LFM signals were generated and analyzed using MATLAB to understand their time-domain and frequency characteristics.

The internship mainly focused on the study and implementation of 1D and 2D beamforming techniques. A 1D linear time-delay beamformer was studied to understand signal alignment and summation in a linear hydrophone array. A 2D planar array beamformer was analyzed using MATLAB by calculating time delays, array factors, normalized beam patterns, and azimuth and elevation cuts. The effects of hydrophone spacing and grating lobes on beamforming performance were also studied. In addition, a 10× polyphase interpolation system using a 60th-order FIR filter was designed and simulated. COE and MEM files were used to transfer filter coefficients and input samples to the Vivado simulation environment.

Overall, the internship improved my understanding of digital signal processing, MATLAB programming, Verilog HDL, and simulation-based design verification. It provided valuable technical experience in analyzing beamforming systems and developing digital signal processing designs. The knowledge gained during this internship provides a strong foundation for further development and future hardware implementation of 1D and 2D beamforming systems for underwater acoustic applications.

REFERENCES

1. MathWorks, *MATLAB Documentation*, The MathWorks, Inc. Available at: https://www.mathworks.com/help/matlab/
2. MathWorks, *Phased Array System Toolbox Documentation*, The MathWorks, Inc. Available at: https://www.mathworks.com/help/phased/
3. MathWorks, *Digital Signal Processing and Signal Analysis Documentation*, The MathWorks, Inc. Available at: https://www.mathworks.com/help/signal/
4. AMD Xilinx, *Vivado Design Suite User Guide: Getting Started*, UG910, Xilinx, 2019.1.
5. AMD Xilinx, *DDS Compiler LogiCORE IP Product Guide*, PG141. Available at: https://docs.amd.com/r/en-US/pg141-dds-compiler
6. MathWorks, *FIR Interpolation: Perform Polyphase FIR Interpolation*. Available at: https://www.mathworks.com/help/dsp/ref/firinterpolation.html
7. John M. Butler and Robert Lowe, “Beamforming Matrix Simplifies Design of Electronically Scanned Antennas,” *Electronic Design*, vol. 9, pp. 170–173, 1961.
8. Harry L. Van Trees, *Optimum Array Processing*, Wiley-Interscience, 2002.
9. Don H. Johnson and Dan E. Dudgeon, *Array Signal Processing: Concepts and Techniques*, Prentice Hall, 1993.
10. Simon Haykin, *Communication Systems*, 4th ed., John Wiley & Sons, 2001.